twenty one Jul

Efficient computation of the r-th root and inverse r-th root of a matrix

By Su Jianlin | July 21, 2025 | 18,782 readers | Kimi (Citation,

In the previous article , "Efficient Calculation of the Square Root and Inverse Square Root of a Matrix," the author discussed...mcsgnStarting with operators, a very elegant method for calculating the square root and inverse square root of a matrix is proposed. Remarkably, after simplification, the final formula no longer shows the original...mcsgnThe form it takes. This inevitably raises deeper questions: What is the more fundamental working principle of this solution? Can it be generalized to any...rWhat are the possibilities of the power root?

Analyzing from this perspective, I was pleasantly surprised to find that we can understand the previous iterative algorithm from a simpler angle, and that this new perspective can be easily generalized to any...rThe power root and the inverse$r$Calculating the power root. We will share the process next.

Previous Recap

set up$\boldsymbol{G}\in\mathrm{R}^{m\times n}$It is any matrix.$\boldsymbol{P}\in\mathrm{R}^{n\times n}$It is true that any eigenvalue exists.$[0,\,1]$The matrix inside, given in the previous article:

$$\boldsymbol{G}_0=\boldsymbol{G},\quad \boldsymbol{P}_0=\boldsymbol{P}$$

$$\boldsymbol{G}_{t+1}=\boldsymbol{G}_t(a_{t+1}\boldsymbol{I}+b_{t+1}\boldsymbol{P}_t+c_{t+1}\boldsymbol{P}_t^2)$$

$$\boldsymbol{P}_{t+1}=(a_{t+1}\boldsymbol{I}+b_{t+1}\boldsymbol{P}_t+c_{t+1}\boldsymbol{P}_t^2)^2\boldsymbol{P}_t$$

$$\lim_{t\to\infty}\boldsymbol{G}_t=\boldsymbol{G}\boldsymbol{P}^{-1/2}$$

Substitution$\boldsymbol{G}=\boldsymbol{P}$It can be obtained$\boldsymbol{P}^{1/2}$Substitute$\boldsymbol{G}=\boldsymbol{I}$It can be obtained$\boldsymbol{P}^{-1/2}$A closer look reveals that the above iterations actually represent the following limit:

$$\prod_{t=0}^{\infty}(a_{t+1}\boldsymbol{I}+b_{t+1}\boldsymbol{P}_t+c_{t+1}\boldsymbol{P}_t^2)=\boldsymbol{P}^{-1/2}$$

Interestingly, proving this limit directly is not complicated; one can directly apply the formula...(2)By taking the square root of both sides and then substituting it into the above equation, we can obtain...

$$\prod_{t=0}^{\infty}(a_{t+1}\boldsymbol{I}+b_{t+1}\boldsymbol{P}_t+c_{t+1}\boldsymbol{P}_t^2)=\prod_{t=0}^{\infty}\boldsymbol{P}_{t+1}^{1/2}\boldsymbol{P}_t^{-1/2}=\lim_{t\to\infty}\boldsymbol{P}_t^{1/2}\boldsymbol{P}_0^{-1/2}=\lim_{t\to\infty}\boldsymbol{P}_t^{1/2}\boldsymbol{P}^{-1/2}$$

Therefore, as long as the sequence $\{\boldsymbol{P}_t\}$ It remains reversible at all times, and its ultimate limit is... $\boldsymbol{I}$ So the limit (3) It is automatically established. As for iteration... (2) How to make $\{\boldsymbol{P}_t\}$ Let's keep these two conditions in mind and discuss them together later.

General form

Let's consider iteration in general.

$$\boldsymbol{G}_0 = \boldsymbol{G}, \quad \boldsymbol{P}_0 = \boldsymbol{P}$$

$$\boldsymbol{G}_{t+1} = \boldsymbol{G}_t (a_{t+1}\boldsymbol{I} + b_{t+1}\boldsymbol{P}_t + c_{t+1}\boldsymbol{P}_t^2)^s$$

$$\boldsymbol{P}_{t+1} = (a_{t+1}\boldsymbol{I} + b_{t+1}\boldsymbol{P}_t + c_{t+1}\boldsymbol{P}_t^2)^r \boldsymbol{P}_t$$

Similarly, if the sequence $\{\boldsymbol{P}_t\}$ It remains reversible at all times, and its ultimate limit is... $\boldsymbol{I}$ Then it can be proven that it is true.

$$\lim_{t\to\infty} \boldsymbol{G}_t = \boldsymbol{G}\boldsymbol{P}^{-s/r}$$

Thus, we obtain a method for finding arbitrary matrices. $-s/r$ The general iterative form of exponentiation. Based on this result, we simply need to choose... $\boldsymbol{G} = \boldsymbol{P}, s = r - 1$ You can get $\boldsymbol{P}^{1/r}$ So we only need to focus on solving it. $0 \sim 1$ The inverse of the power is sufficient.

This makes the question of how to choose the appropriate $\{a_t, b_t, c_t\}$ Let the sequence $\{\boldsymbol{P}_t\}$ To converge as quickly as possible $\boldsymbol{I}$ The faster the convergence speed, the fewer iterations we need to achieve the specified accuracy.

Iteration coefficient

Based on the assumption, $\boldsymbol{P}_0 = \boldsymbol{P}$ It is an eigenvalue that is all $[0, 1]$ The matrix inside, and the target matrix $\boldsymbol{I}$ It is a matrix whose eigenvalues are all 1, so the sequence $\{\boldsymbol{P}_t\}$ In essence, it means that the eigenvalues are arbitrary. $[0, 1]$ The number inside becomes 1 The process, which is actually mcsgn What we've done!

We set $\boldsymbol{X}_t = \boldsymbol{P}_t^{1/r}$, So $\boldsymbol{X}_0 = \boldsymbol{P}^{1/r}$ Similarly, all of them have the same eigenvalue. $[0, 1]$ The matrix inside, and the iterative equation becomes

$$\boldsymbol{X}_{t+1} = a_{t+1}\boldsymbol{X}_t + b_{t+1}\boldsymbol{X}_t^{r+1} + c_{t+1}\boldsymbol{X}_t^{2r+1}$$

The question now becomes how to make$\boldsymbol{X}_0$Turn as quickly as possible$\boldsymbol{I}$This is essentially the same problem we dis cussed in "Newton-Schulz Iteration of the msign Operator (Part 1)" and "Newton-Schulz Iteration of the msign O perator (Part 2)" . The " Part 2 " section provides...$r = 2$It is the theoretically optimal solution, but its solution pro cess and conclusions can be generalized to any...$r$of.

Specifically, we first transform the problem into a scalar iteration:

$$x_{t+1} = f_t(x_t) = a_{t+1}x_t + b_{t+1}x_t^{r+1} + c_{t+1}x_t^{2r+1}$$

Then it is proven that the greedy solution is the optimal solution, and finding the greedy solution becomes solving the equation.

$$\begin{gathered} f_t(l_t) = 1 - \mathcal{E}, \quad f_t(u_t) = 1 + \mathcal{E} \\ f_t(x_{t1}) = 1 + \mathcal{E}, \quad f_t(x_{t2}) = 1 - \mathcal{E} \\ f_t'(x_1) = 0, \quad f_t'(x_2) = 0 \end{gathered}$$

For simplicity,f_tParameterization

$$f_t'(x) = k(x^r - x_1^r)(x^r - x_2^r)$$

Then, just like in the next article , we can use Mathematica to solve it.

Initial Analysis

However, before we proceed with the actual solution, we need to analyze the initialization. In the previous article, "Efficient Calculation of Matrix Square Root and Inverse Square Root," we mentioned that in...$\boldsymbol{P}$Under the assum ption that all eigenvalues are non-negative, we can divide by$\text{tr}(\boldsymbol{P})$Compress all eigenvalues to$[0, 1]$However, this co mpression ratio is often too high; in this article, we will change it to...

$$\boldsymbol{P}_0 = \frac{\boldsymbol{P}}{\sqrt{\text{tr}(\boldsymbol{P}^2)}}$$

We know that$\text{tr}(\boldsymbol{P}^2)$It equals the sum of the squares of all eigenvalues.$\text{tr}(\boldsymbol{P})^2$This is equal to the square of the sum o f all eigenvalues, and holds true for all non-negative eigenvalues.$\text{tr}(\boldsymbol{P}^2) \leq \text{tr}(\boldsymbol{P})^2$Therefore, the above formula provi

des a more compact initial value. Specifically, calculating...$\mathrm{tr}(\boldsymbol{P}^2)$It is not necessary to put$\boldsymbol{P}^2$ We can calculate it exp licitly because we have the identity.

$$\mathrm{tr}(\boldsymbol{P}^2) = \langle \boldsymbol{P}, \boldsymbol{P}^\top \rangle_F$$

Next, we need to analyze how small the eigenvalues need to be processed, which is the same as the initial singular value analysis in "Newton-Schulz Iteration of the msign Operator (Part 1)" . Dividing by $\sqrt{\mathrm{tr}(\boldsymbol{P}^2)}$back,$\boldsymbol{P}_0$ The eigenv alues form a unit vector. If all eigenvalues are equal, then each eigenvalue is...$1/\sqrt{n}$According to the pigeonhole pr inciple, there must generally be a value less than [a certain value].$1/\sqrt{n}$For the sake of conservatism, we are comp atible with the eigenvalues.$0.01/\sqrt{n}$.

Considering a sufficiently large LLM,nWe have arrived.100^2 This level, so we need to be compatible with it.0.0001 Note that this is only...$\boldsymbol{P}_0$ eigenvalues, and$\boldsymbol{X}_0 = \boldsymbol{P}_0^{1/r}$,so$\boldsymbol{X}_0$ We just need to be compatible with$0.0001^{1/r}$This is better thanmcsgn,msignThe situation is more ideal becausemcsgn,msignThe input is$\boldsymbol{X}_0$ We need compatibility$\boldsymbol{X}_0$ Small feat ure values, but the input here is$\boldsymbol{P}_0$ We only need to start from$\boldsymbol{P}_0$ Consider this before proceeding.

Calculation result

Taking all the above considerations into account, our final solution code is as follows:

```
r = 4;
df[x_] = k*(x^r - x1^r) (x^r - x2^r);
f[x_] = Integrate[df[x], {x, 0, x}];
sol[l_, u_] :=
 NSolve[{f[l] == 1 - e, f[x1] == 1 + e, f[x2] == 1 - e, f[u] == 1 + e,
    l < x1 < x2 < u, e > 0, k > 0}, {k, x1, x2, e}]
ff[x_, l_, u_] = f[x]*2/(f[l] + f[u]) // Expand;
lt = 0.0001^(1/r); ut = 1; lambda = 0.1;
While[1 - lt > 0.0001,
 fff[x_] = ff[x, lt, ut] /. sol[Max[lt, lambda*ut], ut][[1]];
 Print[fff[x]];
 lt = fff[lt]; ut = 2 - lt]
f[x] /. Solve[f[1] == 1, k][[1]] /. {x1 -> 1, x2 -> 1}
```

$r = 1 \sim 5$The calculation results are as follows:

r	t	a	b	c
	1	14.2975	-31.2203	18.9214
	2	7.12258	-7.78207	2.35989
1	3	6.9396	-7.61544	2.3195
	4	5.98456	-6.77016	2.12571
	5	3.79109	-4.18664	1.39555
	≥ 6	3	-3	1
	1	7.42487	-18.3958	12.8967
	2	3.48773	-2.33004	0.440469
2	3	2.77661	-2.07064	0.463023
	4	1.99131	-1.37394	0.387593
	≥ 5	15/8	-5/4	3/8
	1	5.05052	-13.5427	10.2579
	2	2.31728	-1.06581	0.144441
3	3	1.79293	-0.913562	0.186699
	4	1.56683	-0.786609	0.220008
	≥ 5	14/9	-7/9	2/9
	1	3.85003	-10.8539	8.61893
4	2	1.80992	-0.587778	0.0647852
	3	1.50394	-0.594516	0.121161
	≥ 4	45/32	-9/16	5/32
	1	3.11194	-8.28217	6.67716
5	2	1.5752	-0.393327	0.0380364
	3	1.3736	-0.44661	0.0911259
	≥ 4	33/25	-11/25	3/25

The convergence value in the final step is determined by...$x_1 = x_2 = 1$and$f(1) = 1$The conclusion is as follows.

Let's test it out .

Here is a simple test code:

```
import numpy as np
import jax.numpy as jnp

coefs = [
    None,
    [
        (14.2975, -31.2203, 18.9214),
        (7.12258, -7.78207, 2.35989),
        (6.9396, -7.61544, 2.3195),
        (5.98456, -6.77016, 2.12571),
        (3.79109, -4.18664, 1.39555),
        (3, -3, 1),
    ],
```

```
    [
        (7.42487, -18.3958, 12.8967),
        (3.48773, -2.33004, 0.440469),
        (2.77661, -2.07064, 0.463023),
        (1.99131, -1.37394, 0.387593),
        (15 / 8, -5 / 4, 3 / 8),
    ],
    [
        (5.05052, -13.5427, 10.2579),
        (2.31728, -1.06581, 0.144441),
        (1.79293, -0.913562, 0.186699),
        (1.56683, -0.786609, 0.220008),
        (14 / 9, -7 / 9, 2 / 9),
    ],
    [
        (3.85003, -10.8539, 8.61893),
        (1.80992, -0.587778, 0.0647852),
        (1.50394, -0.594516, 0.121161),
        (45 / 32, -9 / 16, 5 / 32),
    ],
    [
        (3.11194, -8.28217, 6.67716),
        (1.5752, -0.393327, 0.0380364),
        (1.3736, -0.44661, 0.0911259),
        (33 / 25, -11 / 25, 3 / 25),
    ],
]

def abc(r=1, steps=None, scale=1):
    w, steps = coefs[r], steps or len(coefs[r])
    for a, b, c in w[:steps] + w[-1:] * max(steps - len(w), 0):
        yield a / scale, b / scale**(r + 1), c / scale**(2 * r + 1)

def matmul_invroot(G, P, r, s=1, steps=None, eps=1e-5):
    """return G @ P^(-s/r)
    """
    I = jnp.eye(P.shape[0], dtype=P.dtype)
    P = P / (t := (P * P.mT).sum()**0.5) + eps * I
    for a, b, c in abc(r, steps, 1.001):
        W = a * I + b * P + c * P @ P
        W1, W2 = jnp.linalg.matrix_power(W, s), jnp.linalg.matrix_power(W, r)
        G, P = G @ W1, P @ W2
    return G * t**(-s / r)

def matmul_invroot_by_eigh(G, P, r, s=1):
    """return G @ P^(-s/r)
```

```
    """
    S, Q = jnp.linalg.eigh(P)
    return G @ Q @ jnp.diag(S**(-s / r)) @ jnp.linalg.inv(Q)

d = 1000
s, r = 1, 4
G = np.random.randn(2 * d, d) / d**0.5
P = (x := np.random.randn(d, d) / d**0.5) @ x.T + 0.001 * np.eye(d)

X1 = matmul_invroot_by_eigh(G, P, r, s)
X2 = matmul_invroot(G, P, r, s, eps=0)
jnp.abs(X1 - X2).mean()  # ~= 1e-3

X2 = matmul_invroot(jnp.array(G, dtype='bfloat16'), jnp.array(P, dtype='bfloa
jnp.abs(X1 - X2).mean()  # ~= 2e-3
```

Here are a few points to note. First, enter...$\boldsymbol{P}$The minimum eigenvalue cannot be too small, otherwise the iterative process is extremely prone to exploding, even if we only want to find positive powers.$\boldsymbol{P}^{1/2}$That's also true. This isn't difficult to understand, because...$\sqrt{x}$exist$x = 0$The problem is also quite pathological. Once it "accidentally" reaches the negative half-axis due to error issues, it simply does not exist (real number solution), and the performance of the iteration cannot be predicted at this time.

How small is considered "too small"? Probably around...$\boldsymbol{P}/\sqrt{\text{tr}(\boldsymbol{P}^2)}$The smallest eigenvalue of is clearly not less than the smallest eigenvalue we are considering, that is...0.0001If this cannot be guaranteed, then it is recommended to set it directly.

$$\boldsymbol{P}_0 = \frac{\boldsymbol{P}}{\sqrt{\text{tr}(\boldsymbol{P}^2)}} + \epsilon \cdot \boldsymbol{I}$$

in$\epsilon \sim 0.0001$This will result in a slight loss of precision, but it can significantly increase numerical stability.

Furthermore, the number of iterations generally does not need to exceed the recommended value , especially in low-precision computation scenarios. This is because more iterations increase the likelihood of errors exploding due to accumulated error. In fact, as long as the feature values are within the acceptable range, the recommended number of iterations is sufficient to achieve the desired accuracy, unless we are using fp32 or even higher precision iterations, in which case setting the recommended number is not advisable.`len(coefs[r])`$\epsilon = 0$And use more iterations.`scale=1`

Article Summary

This article generalizes the results of the previous article to any...rThe power root and the inverse$r$The calculation of the exponent yields an arbitrary matrix.$-1/r$A general iterative format for powers.

If you need to cite this article, please refer to:

Su Jianlin. (Jul. 21, 2025). "Efficient Calculation of the r-th Root and Inverse r-th Root of a Matrix" [Blog post]. Retrieved from https://kexue.fm/archives/11175

```
@online{kexuefm-11175,
    title={Efficient Calculation of the r-th Root and Inverse r-Root of a Matrix},
    author={苏剑林},
    year={2025},
    month={Jul},
    url={\url{https://kexue.fm/archives/11175}},
}
```